

```

        document.getElementById('output').innerHTML += "<br/>" +
            ex.sampleProperty;
    }

    function basics2() {
        ex.sampleProperty += 1;
        document.getElementById('output').innerHTML += "<br/>" +
            ex.sampleProperty;
    }

```

Notice that the first letter of each member name is changed from uppercase to lowercase. This transformation is part of the support that JavaScript provides to enable the natural use of the Windows Runtime. Namespaces and class names are Pascal-cased. Member names are camel-cased except for event names, which are all lowercase. See [Using the Windows Runtime in JavaScript](#). The rules for camel casing can be confusing. A series of initial uppercase letters normally appears as lowercase, but if three uppercase letters are followed by a lowercase letter, only the first two letters appear in lowercase: for example, a member named `IDStringKind` appears as `idStringKind`. In Visual Studio, you can build your Windows Runtime component project and then use IntelliSense in your JavaScript project to see the correct casing.

In similar fashion, .NET provides support to enable the natural use of the Windows Runtime in managed code. This is discussed in subsequent sections of this article, and in the articles [Windows Runtime components with C# and Visual Basic](#) and [.NET support for UWP apps and the Windows Runtime](#).

Create a simple user interface

In your JavaScript project, open the `default.html` file and update the body as shown in the following code. This code includes the complete set of controls for the example app and specifies the function names for the click events.

Note When you first run the app, only the Basics1 and Basics2 button are supported.

HTML

```

<body>

    <div id="buttons">
        <button id="button1" >Basics 1</button>
        <button id="button2" >Basics 2</button>

        <button id="runtimeButton1">Runtime 1</button>
        <button id="runtimeButton2">Runtime 2</button>
    </div>

```

```

        <button id="returnsButton1">Returns 1</button>
        <button id="returnsButton2">Returns 2</button>

        <button id="events1Button">Events 1</button>

        <button id="btnAsync">Async</button>
        <button id="btnCancel" disabled="disabled">Cancel Async</button>
        <progress id="primeProg" value="25" max="100" style="color:
yellow;"></progress>
    </div>
    <div id="output">
    </div>
</body>

```

In your JavaScript project, in the css folder, open default.css. Modify the body section as shown, and add styles to control the layout of buttons and the placement of output text.

```

css

body
{
    -ms-grid-columns: 1fr;
    -ms-grid-rows: 1fr 14fr;
    display: -ms-grid;
}

#buttons {
    -ms-grid-rows: 1fr;
    -ms-grid-columns: auto;
    -ms-grid-row-align: start;
}

#output {
    -ms-grid-row: 2;
    -ms-grid-column: 1;
}

```

Now add the event listener registration code by adding a then clause to the processAll call in app.onactivated in default.js. Replace the existing line of code that calls setPromise and change it to the following code:

```

JavaScript

args.setPromise(WinJS.UI.processAll().then(function () {
    var button1 = document.getElementById("button1");
    button1.addEventListener("click", basics1, false);
    var button2 = document.getElementById("button2");
    button2.addEventListener("click", basics2, false);
})));

```

This is a better way to add events to HTML controls than adding a click event handler directly in HTML. See [Create a "Hello, World" app \(JS\)](#).

Build and run the app

Before you build, change the target platform for all projects to Arm, x64, or x86, as appropriate for your computer.

To build and run the solution, choose the F5 key. (If you get a run-time error message stating that SampleComponent is undefined, the reference to the class library project is missing.)

Visual Studio first compiles the class library, and then executes an MSBuild task that runs [Winmdexp.exe \(Windows Runtime Metadata Export Tool\)](#) to create your Windows Runtime component. The component is included in a .winmd file that contains both the managed code and the Windows metadata that describes the code. WinMdExp.exe generates build error messages when you write code that's invalid in a Windows Runtime component, and the error messages are displayed in the Visual Studio IDE. Visual Studio adds your component to the app package (.appx file) for your UWP application, and generates the appropriate manifest.

Choose the Basics 1 button to assign the return value from the static GetAnswer method to the output area, create an instance of the Example class, and display the value of its SampleProperty property in the output area. The output is shown here:

syntax

```
"The answer is 42."  
0
```

Choose the Basics 2 button to increment the value of the SampleProperty property and to display the new value in the output area. Primitive types such as strings and numbers can be used as parameter types and return types, and can be passed between managed code and JavaScript. Because numbers in JavaScript are stored in double-precision floating-point format, they are converted to .NET Framework numeric types.

Note By default, you can set breakpoints only in your JavaScript code. To debug your Visual Basic or C# code, see [Creating Windows Runtime components in C# and Visual Basic](#).

To stop debugging and close your app, switch from the app to Visual Studio, and choose Shift+F5.

Using the Windows Runtime from JavaScript and managed code

The Windows Runtime can be called from either JavaScript or managed code. Windows Runtime objects can be passed back and forth between the two, and events can be handled from either side. However, the ways you use Windows Runtime types in the two environments differ in some details, because JavaScript and .NET support the Windows Runtime differently. The following example demonstrates these differences, using the [Windows.Foundation.Collections.PropertySet](#) class. In this example, you create an instance of the PropertySet collection in managed code and register an event handler to track changes in the collection. Then you add JavaScript code that gets the collection, registers its own event handler, and uses the collection. Finally, you add a method that makes changes to the collection from managed code and shows JavaScript handling a managed exception.

Important In this example, the event is being fired on the UI thread. If you fire the event from a background thread, for example in an async call, you will need to do some extra work in order for JavaScript to handle the event. For more information, see [Raising Events in Windows Runtime components](#).

In the SampleComponent project, add a new **public sealed** class (**Public NotInheritable** class in Visual Basic) named PropertySetStats. The class wraps a PropertySet collection and handles its MapChanged event. The event handler tracks the number of changes of each kind that occur, and the DisplayStats method produces a report that is formatted in HTML. Notice the additional **using** statement (**Imports** statement in Visual Basic); be careful to add this to the existing **using** statements rather than overwriting them.

C#

```
using Windows.Foundation.Collections;

namespace SampleComponent
{
    public sealed class PropertySetStats
    {
        private PropertySet _ps;
        public PropertySetStats()
        {
            _ps = new PropertySet();
            _ps.MapChanged += this.MapChangedHandler;
        }

        public PropertySet PropertySet { get { return _ps; } }

        int[] counts = { 0, 0, 0, 0 };
    }
}
```

```

        private void MapChangedHandler(IObservableMap<string, object>
sender,
        IMapChangedEventArgs<string> args)
        {
            counts[(int)args.CollectionChange] += 1;
        }

        public string DisplayStats()
        {
            StringBuilder report = new StringBuilder("<br/>Number of
changes:<ul>");
            for (int i = 0; i < counts.Length; i++)
            {
                report.Append("<li>" + (CollectionChange)i + ": " +
counts[i] + "</li>");
            }
            return report.ToString() + "</ul>";
        }
    }
}

```

VB

```

Imports System.Text

Public NotInheritable Class PropertySetStats
    Private _ps As PropertySet
    Public Sub New()
        _ps = New PropertySet()
        AddHandler _ps.MapChanged, AddressOf Me.MapChangedHandler
    End Sub

    Public ReadOnly Property PropertySet As PropertySet
        Get
            Return _ps
        End Get
    End Property

    Dim counts() As Integer = {0, 0, 0, 0}
    Private Sub MapChangedHandler(ByVal sender As IObservableMap(Of String,
Object),
        ByVal args As IMapChangedEventArgs(Of String))

        counts(CInt(args.CollectionChange)) += 1
    End Sub

    Public Function DisplayStats() As String
        Dim report As New StringBuilder("<br/>Number of changes:<ul>")
        For i As Integer = 0 To counts.Length - 1
            report.Append("<li>" & CType(i, CollectionChange).ToString() &
                ": " & counts(i) & "</li>")
        Next
        Return report.ToString() & "</ul>"
    End Function
End Class

```

```
End Function
End Class
```

The event handler follows the familiar .NET Framework event pattern, except that the sender of the event (in this case, the `PropertySet` object) is cast to the `IObservableMap<string, object>` interface (`IObservableMap(Of String, Object)` in Visual Basic), which is an instantiation of the Windows Runtime interface `IObservableMap<K, V>`. (You can cast the sender to its type if necessary.) Also, the event arguments are presented as an interface rather than as an object.

In the `default.js` file, add the `Runtime1` function as shown. This code creates a `PropertySetStats` object, gets its `PropertySet` collection, and adds its own event handler, the `onMapChanged` function, to handle the `MapChanged` event. After making changes to the collection, `runtime1` calls the `DisplayStats` method to show a summary of change types.

JavaScript

```
var propertysetstats;

function runtime1() {
    document.getElementById('output').innerHTML = "";

    propertysetstats = new SampleComponent.PropertySetStats();
    var propertyset = propertysetstats.propertySet;

    propertyset.addEventListener("mapchanged", onMapChanged);

    propertyset.insert("FirstProperty", "First property value");
    propertyset.insert("SuperfluousProperty", "Unnecessary property value");
    propertyset.insert("AnotherProperty", "A property value");

    propertyset.insert("SuperfluousProperty", "Altered property value");
    propertyset.remove("SuperfluousProperty");

    document.getElementById('output').innerHTML +=
        propertysetstats.displayStats();
}

function onMapChanged(change) {
    var result
    switch (change.collectionChange) {
        case Windows.Foundation.Collections.CollectionChange.reset:
            result = "All properties cleared";
            break;
        case Windows.Foundation.Collections.CollectionChange.itemInserted:
            result = "Inserted " + change.key + ": '" +
                change.target.lookup(change.key) + "'";
            break;
        case Windows.Foundation.Collections.CollectionChange.itemRemoved:
```

```

        result = "Removed " + change.key;
        break;
    case Windows.Foundation.Collections.CollectionChange.itemChanged:
        result = "Changed " + change.key + " to '" +
            change.target.lookup(change.key) + "'";
        break;
    default:
        break;
}

document.getElementById('output').innerHTML +=
    "<br/>" + result;
}

```

The way you handle Windows Runtime events in JavaScript is very different from the way you handle them in .NET Framework code. The JavaScript event handler takes only one argument. When you view this object in the Visual Studio debugger, the first property is the sender. The members of the event argument interface also appear directly on this object.

To run the app, choose the F5 key. If the class is not sealed, you get the error message, "Exporting unsealed type 'SampleComponent.Example' is not currently supported. Please mark it as sealed."

Choose the **Runtime 1** button. The event handler displays changes as elements are added or changed, and at the end the DisplayStats method is called to produce a summary of counts. To stop debugging and close the app, switch back to Visual Studio and choose Shift+F5.

To add two more items to the PropertySet collection from managed code, add the following code to the PropertySetStats class:

C#

```

public void AddMore()
{
    _ps.Add("NewProperty", "New property value");
    _ps.Add("AnotherProperty", "A property value");
}

```

VB

```

Public Sub AddMore()
    _ps.Add("NewProperty", "New property value")
    _ps.Add("AnotherProperty", "A property value")
End Sub

```

This code highlights another difference in the way you use Windows Runtime types in the two environments. If you type this code yourself, you'll notice that IntelliSense doesn't show the insert method you used in the JavaScript code. Instead, it shows the Add method commonly seen on collections in .NET. This is because some commonly used collection interfaces have different names but similar functionality in the Windows Runtime and .NET. When you use these interfaces in managed code, they appear as their .NET Framework equivalents. This is discussed in [Windows Runtime components with C# and Visual Basic](#). When you use the same interfaces in JavaScript, the only change from the Windows Runtime is that uppercase letters at the beginning of member names become lowercase.

Finally, to call the AddMore method with exception handling, add the runtime2 function to default.js.

JavaScript

```
function runtime2() {
    try {
        propertysetstats.addMore();
    }
    catch(ex) {
        document.getElementById('output').innerHTML +=
            "<br/><b>" + ex + "<br/>";
    }

    document.getElementById('output').innerHTML +=
        propertysetstats.displayStats();
}
```

Add the event handler registration code the same way you did previously.

JavaScript

```
var runtimeButton1 = document.getElementById("runtimeButton1");
runtimeButton1.addEventListener("click", runtime1, false);
var runtimeButton2 = document.getElementById("runtimeButton2");
runtimeButton2.addEventListener("click", runtime2, false);
```

To run the app, choose the F5 key. Choose **Runtime 1** and then **Runtime 2**. The JavaScript event handler reports the first change to the collection. The second change, however, has a duplicate key. Users of .NET Framework dictionaries expect the Add method to throw an exception, and that is what happens. JavaScript handles the .NET exception.

Note You can't display the exception's message from JavaScript code. The message text is replaced by a stack trace. For more information, see "Throwing exceptions" in

By contrast, when JavaScript called the insert method with a duplicate key, the value of the item was changed. This difference in behavior is due to the different ways that JavaScript and .NET support the Windows Runtime, as explained in [Windows Runtime components with C# and Visual Basic](#).

Returning managed types from your component

As discussed previously, you can pass native Windows Runtime types back and forth freely between your JavaScript code and your C# or Visual Basic code. Most of the time, the type names and member names will be the same in both cases (except that the member names start with lowercase letters in JavaScript). However, in the preceding section, the PropertySet class appeared to have different members in managed code. (For example, in JavaScript you called the insert method, and in the .NET code you called the Add method.) This section explores the way those differences affect .NET Framework types passed to JavaScript.

In addition to returning Windows Runtime types that you created in your component or passed to your component from JavaScript, you can return a managed type, created in managed code, to JavaScript as if it were the corresponding Windows Runtime type. Even in the first, simple example of a runtime class, the parameters and return types of the members were Visual Basic or C# primitive types, which are .NET Framework types. To demonstrate this for collections, add the following code to the Example class, to create a method that returns a generic dictionary of strings, indexed by integers:

C#

```
public static IDictionary<int, string> GetMapOfNames()
{
    Dictionary<int, string> retval = new Dictionary<int, string>();
    retval.Add(1, "one");
    retval.Add(2, "two");
    retval.Add(3, "three");
    retval.Add(42, "forty-two");
    retval.Add(100, "one hundred");
    return retval;
}
```

VB

```

Public Shared Function GetMapOfNames() As IDictionary(Of Integer, String)
    Dim retval As New Dictionary(Of Integer, String)
    retval.Add(1, "one")
    retval.Add(2, "two")
    retval.Add(3, "three")
    retval.Add(42, "forty-two")
    retval.Add(100, "one hundred")
    Return retval
End Function

```

Notice that the dictionary must be returned as an interface that is implemented by `Dictionary<TKey, TValue>`, and that maps to a Windows Runtime interface. In this case, the interface is `IDictionary<int, string>` (`IDictionary(Of Integer, String)` in Visual Basic). When the Windows Runtime type `IMap<int, string>` is passed to managed code, it appears as `IDictionary<int, string>`, and the reverse is true when the managed type is passed to JavaScript.

Important When a managed type implements multiple interfaces, JavaScript uses the interface that appears first in the list. For example, if you return `Dictionary<int, string>` to JavaScript code, it appears as `IDictionary<int, string>` no matter which interface you specify as the return type. This means that if the first interface doesn't include a member that appears on later interfaces, that member isn't visible to JavaScript.

To test the new method and use the dictionary, add the `returns1` and `returns2` functions to `default.js`:

JavaScript

```

var names;

function returns1() {
    names = SampleComponent.Example.getMapOfNames();
    document.getElementById('output').innerHTML = showMap(names);
}

var ct = 7;

function returns2() {
    if (!names.hasKey(17)) {
        names.insert(43, "forty-three");
        names.insert(17, "seventeen");
    }
    else {
        var err = names.insert("7", ct++);
        names.insert("forty", "forty");
    }
}

```

```

        document.getElementById('output').innerHTML = showMap(names);
    }

    function showMap(map) {
        var item = map.first();
        var retval = "<ul>";

        for (var i = 0, len = map.size; i < len; i++) {
            retval += "<li>" + item.current.key + ": " + item.current.value + "
</li>";
            item.moveToNext();
        }
        return retval + "</ul>";
    }
}

```

Add the event registration code to the same then block as the other event registration code:

JavaScript

```

var returnsButton1 = document.getElementById("returnsButton1");
returnsButton1.addEventListener("click", returns1, false);
var returnsButton2 = document.getElementById("returnsButton2");
returnsButton2.addEventListener("click", returns2, false);

```

There are a few interesting things to observe about this JavaScript code. First of all, it includes a showMap function to display the contents of the dictionary in HTML. In the code for showMap, notice the iteration pattern. In .NET, there's no First method on the generic IDictionary interface, and the size is returned by a Count property rather than by a Size method. To JavaScript, IDictionary<int, string> appears to be the Windows Runtime type IMap<int, string>. (See the [IMap<K,V>](#) interface.)

In the returns2 function, as in earlier examples, JavaScript calls the Insert method (insert in JavaScript) to add items to the dictionary.

To run the app, choose the F5 key. To create and display the initial contents of the dictionary, choose the **Returns 1** button. To add two more entries to the dictionary, choose the **Returns 2** button. Notice that the entries are displayed in order of insertion, as you would expect from Dictionary<TKey, TValue>. If you want them sorted, you can return a SortedDictionary<int, string> from GetMapOfNames. (The PropertySet class used in earlier examples has a different internal organization from Dictionary<TKey, TValue>.)

Of course, JavaScript is not a strongly typed language, so using strongly typed generic collections can lead to some surprising results. Choose the **Returns 2** button again. JavaScript obligingly coerces the "7" to a numeric 7, and the numeric 7 that's stored in ct

to a string. And it coerces the string "forty" to zero. But that's only the beginning. Choose the **Returns 2** button a few more times. In managed code, the Add method would generate duplicate key exceptions, even if the values were cast to the correct types. In contrast, the Insert method updates the value associated with an existing key and returns a Boolean value that indicates whether a new key was added to the dictionary. This is why the value associated with the key 7 keeps changing.

Another unexpected behavior: If you pass an unassigned JavaScript variable as a string argument, what you get is the string "undefined". In short, be careful when you pass .NET Framework collection types to your JavaScript code.

Note If you have large quantities of text to concatenate, you can do it more efficiently by moving the code into a .NET Framework method and using the StringBuilder class, as shown in the showMap function.

Although you can't expose your own generic types from a Windows Runtime component, you can return .NET Framework generic collections for Windows Runtime classes by using code such as the following:

C#

```
public static object GetListOfThis(object obj)
{
    Type target = obj.GetType();
    return Activator.CreateInstance(typeof(List<>).MakeGenericType(target));
}
```

VB

```
Public Shared Function GetListOfThis(obj As Object) As Object
    Dim target As Type = obj.GetType()
    Return Activator.CreateInstance(GetType(List(Of
)).MakeGenericType(target))
End Function
```

List<T> implements IList<T>, which appears as the Windows Runtime type IVector<T> in JavaScript.

Declaring events

You can declare events by using the standard .NET Framework event pattern or other patterns used by the Windows Runtime. The .NET Framework supports equivalence between the System.EventHandler<TEventArgs> delegate and the Windows Runtime

EventHandler<T> delegate, so using EventHandler<TEventArgs> is a good way to implement the standard .NET Framework pattern. To see how this works, add the following pair of classes to the SampleComponent project:

C#

```
namespace SampleComponent
{
    public sealed class Eventful
    {
        public event EventHandler<TestEventArgs> Test;
        public void OnTest(string msg, long number)
        {
            EventHandler<TestEventArgs> temp = Test;
            if (temp != null)
            {
                temp(this, new TestEventArgs()
                {
                    Value1 = msg,
                    Value2 = number
                });
            }
        }
    }

    public sealed class TestEventArgs
    {
        public string Value1 { get; set; }
        public long Value2 { get; set; }
    }
}
```

VB

```
Public NotInheritable Class Eventful
    Public Event Test As EventHandler(Of TestEventArgs)
    Public Sub OnTest(ByVal msg As String, ByVal number As Long)
        RaiseEvent Test(Me, New TestEventArgs() With {
            .Value1 = msg,
            .Value2 = number
        })
    End Sub
End Class

Public NotInheritable Class TestEventArgs
    Public Property Value1 As String
    Public Property Value2 As Long
End Class
```

When you expose an event in the Windows Runtime, the event argument class inherits from `System.Object`. It doesn't inherit from `System.EventArgs`, as it would in .NET, because `EventArgs` is not a Windows Runtime type.

If you declare custom event accessors for your event (**Custom** keyword in Visual Basic), you must use the Windows Runtime event pattern. See [Custom events and event accessors in Windows Runtime components](#).

To handle the `Test` event, add the `events1` function to `default.js`. The `events1` function creates an event handler function for the `Test` event, and immediately invokes the `OnTest` method to raise the event. If you place a breakpoint in the body of the event handler, you can see that the object passed to the single parameter includes the source object and both members of `TestEventArgs`.

JavaScript

```
var ev;

function events1() {
    ev = new SampleComponent.Eventful();
    ev.addEventListener("test", function (e) {
        document.getElementById('output').innerHTML = e.value1;
        document.getElementById('output').innerHTML += "<br/>" + e.value2;
    });
    ev.onTest("Number of feet in a mile:", 5280);
}
```

Add the event registration code to the same then block as the other event registration code:

JavaScript

```
var events1Button = document.getElementById("events1Button");
events1Button.addEventListener("click", events1, false);
```

Exposing asynchronous operations

The .NET Framework has a rich set of tools for asynchronous processing and parallel processing, based on the `Task` and generic `Task<TResult>` classes. To expose task-based asynchronous processing in a Windows Runtime component, use the Windows Runtime interfaces `IAsyncAction`, `IAsyncActionWithProgress<TProgress>`, `IAsyncOperation<TResult>`, and `IAsyncOperationWithProgress<TResult, TProgress>`. (In the Windows Runtime, operations return results, but actions do not.)

This section demonstrates a cancelable asynchronous operation that reports progress and returns results. The `GetPrimesInRangeAsync` method uses the [AsyncInfo](#) class to generate a task and to connect its cancellation and progress-reporting features to a `WinJS.Promise` object. Begin by adding the `GetPrimesInRangeAsync` method to the example class:

C#

```
using System.Runtime.InteropServices.WindowsRuntime;
using Windows.Foundation;

public static IAsyncOperationWithProgress<IList<long>, double>
GetPrimesInRangeAsync(long start, long count)
{
    if (start < 2 || count < 1) throw new ArgumentException();

    return AsyncInfo.Run<IList<long>, double>((token, progress) =>

        Task.Run<IList<long>>(() =>
        {
            List<long> primes = new List<long>();
            double onePercent = count / 100;
            long ctProgress = 0;
            double nextProgress = onePercent;

            for (long candidate = start; candidate < start + count;
candidate++)
            {
                ctProgress += 1;
                if (ctProgress >= nextProgress)
                {
                    progress.Report(ctProgress / onePercent);
                    nextProgress += onePercent;
                }
                bool isPrime = true;
                for (long i = 2, limit = (long)Math.Sqrt(candidate); i <=
limit; i++)
                {
                    if (candidate % i == 0)
                    {
                        isPrime = false;
                        break;
                    }
                }
                if (isPrime) primes.Add(candidate);

                token.ThrowIfCancellationRequested();
            }
            progress.Report(100.0);
            return primes;
        }, token)
    );
}
```

VB

```
Imports System.Runtime.InteropServices.WindowsRuntime

Public Shared Function GetPrimesInRangeAsync(ByVal start As Long, ByVal
count As Long)
As IAsyncOperationWithProgress(Of IList(Of Long), Double)

    If (start < 2 Or count < 1) Then Throw New ArgumentException()

    Return AsyncInfo.Run(Of IList(Of Long), Double)( _
        Function(token, prog)
            Return Task.Run(Of IList(Of Long))(_
                Function()
                    Dim primes As New List(Of Long)
                    Dim onePercent As Long = count / 100
                    Dim ctProgress As Long = 0
                    Dim nextProgress As Long = onePercent

                    For candidate As Long = start To start + count - 1
                        ctProgress += 1

                        If ctProgress >= nextProgress Then
                            prog.Report(ctProgress / onePercent)
                            nextProgress += onePercent
                        End If

                        Dim isPrime As Boolean = True
                        For i As Long = 2 To CLng(Math.Sqrt(candidate))
                            If (candidate Mod i) = 0 Then
                                isPrime = False
                                Exit For
                            End If
                        Next

                        If isPrime Then primes.Add(candidate)

                        token.ThrowIfCancellationRequested()
                    Next
                    prog.Report(100.0)
                    Return primes
                End Function, token)
        End Function)
End Function
```

GetPrimesInRangeAsync is a very simple prime number finder, and that's by design. The focus here is on implementing an asynchronous operation, so simplicity is important, and a slow implementation is an advantage when we're demonstrating cancellation. GetPrimesInRangeAsync finds primes by brute force: It divides a candidate by all the

integers that are less than or equal to its square root, rather than using only the prime numbers. Stepping through this code:

- Before starting an asynchronous operation, perform housekeeping activities such as validating parameters and throwing exceptions for invalid input.
- The key to this implementation is the [AsyncInfo.Run<TResult, TProgress> \(Func<CancellationToken, IProgress<TProgress>, Task<TResult>>\)](#) method, and the delegate that is the method's only parameter. The delegate must accept a cancellation token and an interface for reporting progress, and must return a started task that uses those parameters. When JavaScript calls the `GetPrimesInRangeAsync` method, the following steps occur (not necessarily in the order given here):
 - The [WinJS.Promise](#) object supplies functions to process the returned results, react to cancellation, and handle progress reports.
 - The `AsyncInfo.Run` method creates a cancellation source and an object that implements the `IProgress<T>` interface. To the delegate, it passes both a [CancellationToken](#) token from the cancellation source, and the [IProgress<T>](#) interface.

Note If the Promise object doesn't supply a function to react to cancellation, `AsyncInfo.Run` still passes a cancelable token, and cancellation can still occur. If the Promise object doesn't supply a function to handle progress updates, `AsyncInfo.Run` still supplies an object that implements `IProgress<T>`, but its reports are ignored.

- The delegate uses the [Task.Run<TResult>\(Func<TResult>, CancellationToken\)](#) method to create a started task that uses the token and the progress interface. The delegate for the started task is provided by a lambda function that computes the desired result. More about that in a moment.
- The `AsyncInfo.Run` method creates an object that implements the [IAsyncOperationWithProgress<TResult, TProgress>](#) interface, connects the Windows Runtime cancellation mechanism with the token source, and connects the Promise object's progress-reporting function with the `IProgress<T>` interface.
- The `IAsyncOperationWithProgress<TResult, TProgress>` interface is returned to JavaScript.

- The lambda function that is represented by the started task doesn't take any arguments. Because it's a lambda function, it has access to the token and the IProgress interface. Each time a candidate number is evaluated, the lambda function:
 - Checks to see whether the next percentage point of progress has been reached. If it has, the lambda function calls the IProgress<T>.Report method, and the percentage is passed through to the function that the Promise object specified for reporting progress.
 - Uses the cancellation token to throw an exception if the operation has been canceled. If the [IAsyncInfo.Cancel](#) method (which the IAsyncOperationWithProgress<TResult, TProgress> interface inherits) has been called, the connection that the AsyncInfo.Run method set up ensures that the cancellation token is notified.
- When the lambda function returns the list of prime numbers, the list is passed to the function that the WinJS.Promise object specified for processing the results.

To create the JavaScript promise and set up the cancellation mechanism, add the asyncRun and asyncCancel functions to default.js.

JavaScript

```
var resultAsync;
function asyncRun() {
    document.getElementById('output').innerHTML = "Retrieving prime numbers.";
    btnAsync.disabled = "disabled";
    btnCancel.disabled = "";

    resultAsync =
SampleComponent.Example.getPrimesInRangeAsync(1000000000001, 2500).then(
    function (primes) {
        for (i = 0; i < primes.length; i++)
            document.getElementById('output').innerHTML += " " +
primes[i];

        btnCancel.disabled = "disabled";
        btnAsync.disabled = "";
    },
    function () {
        document.getElementById('output').innerHTML += " --
getPrimesInRangeAsync was canceled. -- ";

        btnCancel.disabled = "disabled";
        btnAsync.disabled = "";
    },
    function (prog) {
        document.getElementById('primeProg').value = prog;
    }
}
```

```

    );
}

function asyncCancel() {
    resultAsync.cancel();
}

```

Don't forget the event registration code the same as you did previously.

JavaScript

```

var btnAsync = document.getElementById("btnAsync");
btnAsync.addEventListener("click", asyncRun, false);
var btnCancel = document.getElementById("btnCancel");
btnCancel.addEventListener("click", asyncCancel, false);

```

By calling the asynchronous `GetPrimesInRangeAsync` method, the `asyncRun` function creates a `WinJS.Promise` object. The object's `then` method takes three functions that process the returned results, react to errors (including cancellation), and handle progress reports. In this example, the returned results are printed in the output area. Cancellation or completion resets the buttons that launch and cancel the operation. Progress reporting updates the progress control.

The `asyncCancel` function just calls the `cancel` method of the `WinJS.Promise` object.

To run the app, choose the `F5` key. To start the asynchronous operation, choose the **Async** button. What happens next depends on how fast your computer is. If the progress bar zips to completion before you have time to blink, increase the size of the starting number that is passed to `GetPrimesInRangeAsync` by one or more factors of ten. You can fine-tune the duration of the operation by increasing or decreasing the count of numbers to test, but adding zeros in the middle of the starting number will have a bigger impact. To cancel the operation, choose the **Cancel Async** button.

Related topics

- [.NET for UWP apps overview](#)
- [.NET for UWP apps](#)

Raising events in Windows Runtime components

Article • 10/20/2022

ⓘ Note

For more info about raising events in a [C++/WinRT](#) Windows Runtime Component, see [Author events in C++/WinRT](#).

If your Windows Runtime component raises an event of a user-defined delegate type on a background thread (worker thread), and you want JavaScript to be able to receive the event, then you can implement and/or raise it in any one of these ways.

- (Option 1) Raise the event through the [Windows.UI.Core.CoreDispatcher](#) to marshal the event to the JavaScript thread context. Although typically this is the best option, in some scenarios it might not provide the fastest performance.
- (Option 2) Use [Windows.Foundation.EventHandler<Object>](#) (but lose the event type information). If Option 1 is not feasible, or if its performance is not adequate, then this is a good second choice provided that loss of type information is acceptable. If you're authoring a C# Windows Runtime Component, then the [Windows.Foundation.EventHandler<Object>](#) type is not available; instead, that type is projected to [System.EventHandler](#), so you should use that instead.
- (Option 3) Create your own proxy and stub for the component. This option is the most difficult to implement, but it preserves type information and might provide better performance compared to Option 1 in demanding scenarios.

If you just raise an event on a background thread without using one of these options, a JavaScript client will not receive the event.

Background

All Windows Runtime components and apps are fundamentally COM objects, no matter what language you use to create them. In the Windows API, most of the components are agile COM objects that can communicate equally well with objects on the background thread and on the UI thread. If a COM object can't be made agile, then it requires helper objects known as proxies and stubs to communicate with other COM objects across the UI thread-background thread boundary. (In COM terms, this is known as communication between thread apartments.)

Most of the objects in the Windows API are either agile or have proxies and stubs built in. However, proxies and stubs can't be created for generic types such as `Windows.Foundation.TypedEventHandler<TSender, TResult>` because they are not complete types until you provide the type argument. It's only with JavaScript clients that the lack of proxies or stubs becomes an issue, but if you want your component to be usable from JavaScript as well as from C++ or a .NET language, then you must use one of the following three options.

(Option 1) Raise the event through the CoreDispatcher

You can send events of any user-defined delegate type by using the `Windows.UI.Core.CoreDispatcher`, and JavaScript will be able to receive them. If you are unsure which option to use, try this one first. If latency between the event firing and the event handling becomes an issue, then try one of the other options.

The following example shows how to use the `CoreDispatcher` to raise a strongly-typed event. Notice that the type argument is `Toast`, not `Object`.

C#

```
public event EventHandler<Toast> ToastCompletedEvent;
private void OnToastCompleted(Toast args)
{
    var completedEvent = ToastCompletedEvent;
    if (completedEvent != null)
    {
        completedEvent(this, args);
    }
}

public void MakeToastWithDispatcher(string message)
{
    Toast toast = new Toast(message);
    // Assume you have a CoreDispatcher at class scope.
    // Initialize it here, then use it from the background thread.
    var window = Windows.UI.Core.CoreWindow.GetForCurrentThread();
    m_dispatcher = window.Dispatcher;

    Task.Run( () =>
    {
        if (ToastCompletedEvent != null)
        {
            m_dispatcher.RunAsync(CoreDispatcherPriority.Normal,
                new DispatchedHandler(() =>
                {
                    this.OnToastCompleted(toast);
                })); // end m_dispatcher.RunAsync
        }
    });
}
```

```
    }  
    }); // end Task.Run  
}
```

(Option 2) Use `EventHandler<Object>` but lose type information

ⓘ Note

If you're authoring a C# Windows Runtime Component, then the `Windows.Foundation.EventHandler<Object>` type is not available; instead, that type is projected to [System.EventHandler](#), so you should use that instead.

Another way to send an event from a background thread is to use `Windows.Foundation.EventHandler<Object>` as the type of the event. Windows provides this concrete instantiation of the generic type and provides a proxy and stub for it. The downside is that the type information of your event args and sender is lost. C++ and .NET clients must know through documentation what type to cast back to when the event is received. JavaScript clients don't need the original type information. They find the arg properties, based on their names in the metadata.

This example shows how to use `Windows.Foundation.EventHandler<Object>` in C#:

C#

```
public sealed Class1  
{  
    // Declare the event  
    public event EventHandler<Object> ToastCompletedEvent;  
  
    // Raise the event  
    public async void MakeToast(string message)  
    {  
        Toast toast = new Toast(message);  
        // Fire the event from a background thread to allow this thread to  
        continue  
        Task.Run(() =>  
        {  
            if (ToastCompletedEvent != null)  
            {  
                OnToastCompleted(toast);  
            }  
        });  
    }  
}
```

```

private void OnToastCompleted(Toast args)
{
    var completedEvent = ToastCompletedEvent;
    if (completedEvent != null)
    {
        completedEvent(this, args);
    }
}
}

```

You consume this event on the JavaScript side like this:

JavaScript

```

toastCompletedEventHandler: function (event) {
    var toastType = event.toast.toastType;
    document.getElementById("toasterOutput").innerHTML = "<p>Made " +
toastType + " toast</p>";
}

```

(Option 3) Create your own proxy and stub

For potential performance gains on user-defined event types that have fully-preserved type information, you have to create your own proxy and stub objects and embed them in your app package. Typically, you have to use this option only in rare situations where neither of the other two options are adequate. Also, there is no guarantee that this option will provide better performance than the other two options. Actual performance depends on many factors. Use the Visual Studio profiler or other profiling tools to measure actual performance in your application and determine whether the event is in fact a bottleneck.

The rest of this article shows how to use C# to create a basic Windows Runtime component, and then use C++ to create a DLL for the proxy and stub that will enable JavaScript to consume a `Windows.Foundation.TypedEventHandler<TSender, TResult>` event that's raised by the component in an async operation. (You can also use C++ or Visual Basic to create the component. The steps that are related to creating the proxies and stubs are the same.) This walkthrough is based on [Creating a Windows Runtime in-process component sample \(C++/CX\)](#) and helps explain its purposes.

This walkthrough has these parts.

- Here you will create two basic Windows Runtime classes. One class exposes an event of type `Windows.Foundation.TypedEventHandler<TSender, TResult>` and the

other class is the type that's returned to JavaScript as the argument for TValue.

These classes can't communicate with JavaScript until you complete the later steps.

- This app activates the main class object, calls a method, and handles an event that's raised by the Windows Runtime component.
- These are required by the tools that generate the proxy and stub classes.
- You then use the IDL file to generate the C source code for the proxy and stub.
- Register the proxy-stub objects so that the COM runtime can find them, and reference the proxy-stub DLL in the app project.

To create the Windows Runtime component

In Visual Studio, on the menu bar, choose **File > New Project**. In the **New Project** dialog box, expand **JavaScript > Universal Windows** and then select **Blank App**. Name the project **ToasterApplication** and then choose the **OK** button.

Add a C# Windows Runtime component to the solution: In Solution Explorer, open the shortcut menu for the solution and then choose **Add > New Project**. Expand **Visual C# > Microsoft Store** and then select **Windows Runtime Component**. Name the project **ToasterComponent** and then choose the **OK** button. **ToasterComponent** will be the root namespace for the components you will create in later steps.

In Solution Explorer, open the shortcut menu for the solution and then choose **Properties**. In the **Property Pages** dialog box, select **Configuration Properties** in the left pane, and then at the top of the dialog box, set **Configuration** to **Debug** and **Platform** to **x86, x64, or ARM**. Choose the **OK** button.

Important Platform = Any CPU won't work because it's not valid for the native-code Win32 DLL that you'll add to the solution later.

In Solution Explorer, rename **class1.cs** to **ToasterComponent.cs** so that it matches the name of the project. Visual Studio automatically renames the class in the file to match the new file name.

In the **.cs** file, add a **using** directive for the **Windows.Foundation** namespace to bring **TypedEventHandler** into scope.

When you require proxies and stubs, your component must use interfaces to expose its public members. In **ToasterComponent.cs**, define an interface for the toaster, and another one for the Toast that the toaster produces.

Note In C# you can skip this step. Instead, first create a class, and then open its shortcut menu and choose **Refactor > Extract Interface**. In the code that's generated, manually give the interfaces public accessibility.

C#

```
public interface IToaster
{
    void MakeToast(String message);
    event TypedEventHandler<Toaster, Toast> ToastCompletedEvent;
}

public interface IToast
{
    String ToastType { get; }
}
```

The IToast interface has a string that can be retrieved to describe the type of toast. The IToaster interface has a method to make toast, and an event to indicate that the toast is made. Because this event returns the particular piece (that is, type) of toast, it's known as a typed event.

Next, we need classes that implement these interfaces, and are public and sealed so that they are accessible from the JavaScript app that you'll program later.

C#

```
public sealed class Toast : IToast
{
    private string _toastType;

    public string ToastType
    {
        get
        {
            return _toastType;
        }
    }

    internal Toast(String toastType)
    {
        _toastType = toastType;
    }
}

public sealed class Toaster : IToaster
{
    public event TypedEventHandler<Toaster, Toast>
ToastCompletedEvent;

    private void OnToastCompleted(Toast args)
    {
        var completedEvent = ToastCompletedEvent;
        if (completedEvent != null)
        {
            completedEvent(this, args);
        }
    }
}
```

```

    }
}

public void MakeToast(string message)
{
    Toast toast = new Toast(message);
    // Fire the event from a thread-pool thread to enable this
    thread to continue
    Windows.System.Threading.ThreadPool.RunAsync(
        (IAsyncAction action) =>
        {
            if (ToastCompletedEvent != null)
            {
                OnToastCompleted(toast);
            }
        });
}
}

```

In the preceding code, we create the toast and then spin up a thread-pool work item to fire the notification. Although the IDE might suggest that you apply the `await` keyword to the async call, it isn't necessary in this case because the method doesn't do any work that depends on the results of the operation.

Note The async call in the preceding code uses `ThreadPool.RunAsync` solely to demonstrate a simple way to fire the event on a background thread. You could write this particular method as shown in the following example, and it would work fine because the .NET Task scheduler automatically marshals async/await calls back to the UI thread.

C#

```

public async void MakeToast(string message)
{
    Toast toast = new Toast(message)
    await Task.Delay(new Random().Next(1000));
    OnToastCompleted(toast);
}

```

If you build the project now, it should build cleanly.

To program the JavaScript app

Now we can add a button to the JavaScript app to cause it to use the class we just defined to make toast. Before we do this, we must add a reference to the `ToasterComponent` project we just created. In Solution Explorer, open the shortcut menu for the `ToasterApplication` project, choose **Add > References**, and then choose the **Add New Reference** button. In the Add Reference dialog box, in the left pane under Solution,

select the component project, and then in the middle pane, select `ToasterComponent`. Choose the **OK** button.

In Solution Explorer, open the shortcut menu for the `ToasterApplication` project and then choose **Set as Startup Project**.

At the end of the `default.js` file, add a namespace to contain the functions to call the component and be called back by it. The namespace will have two functions, one to make toast and one to handle the toast-complete event. The implementation of `makeToast` creates a `Toaster` object, registers the event handler, and makes the toast. So far, the event handler doesn't do much, as shown here:

JavaScript

```
WinJS.Namespace.define("ToasterApplication"), {
    makeToast: function () {

        var toaster = new ToasterComponent.Toaster();
        //toaster.addEventListener("ontoastcompletedevent",
        ToasterApplication.toastCompletedEventHandler);
        toaster.ontoastcompletedevent =
        ToasterApplication.toastCompletedEventHandler;
        toaster.makeToast("Peanut Butter");
    },

    toastCompletedEventHandler: function(event) {
        // The sender of the event (the delegate's first type parameter)
        // is mapped to event.target. The second argument of the delegate
        // is contained in event, which means in this case event is a
        // Toast class, with a toastType string.
        var toastType = event.toastType;

        document.getElementById('toastOutput').innerHTML = "<p>Made " +
toastType + " toast</p>";
    },
});
```

The `makeToast` function must be hooked up to a button. Update `default.html` to include a button and some space to output the result of making toast:

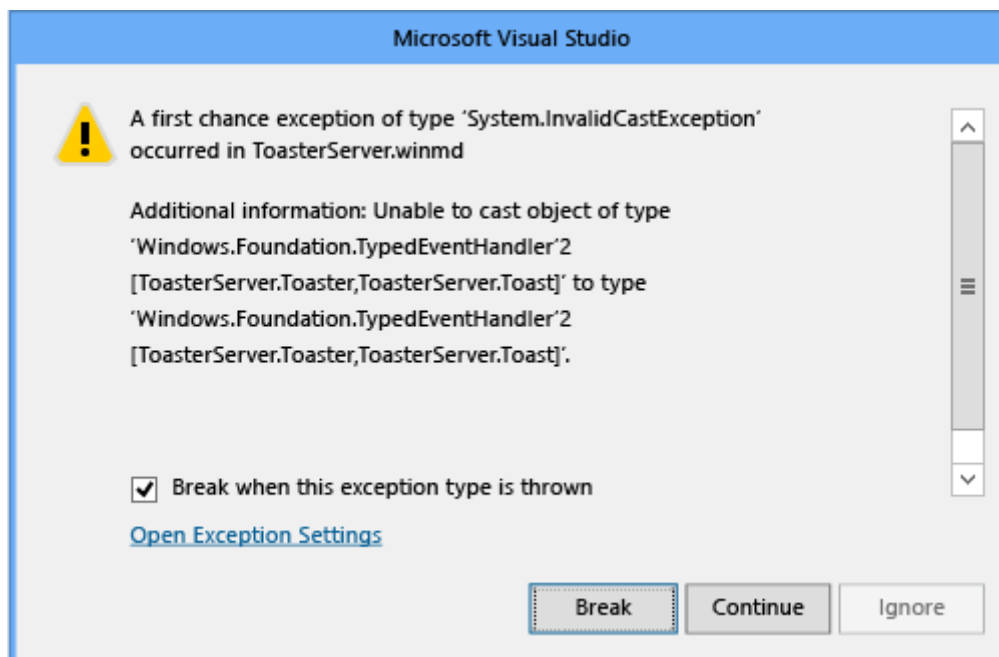
HTML

```
<body>
    <h1>Click the button to make toast</h1>
    <button onclick="ToasterApplication.makeToast()">Make Toast!
</button>
    <div id="toastOutput">
        <p>No Toast Yet...</p>
```

```
</div>  
</body>
```

If we weren't using a TypedEventHandler, we would now be able to run the app on the local machine and click the button to make toast. But in our app, nothing happens. To find out why, let's debug the managed code that fires the ToastCompletedEvent. Stop the project, and then on the menu bar, choose **Debug > Toaster Application properties**. Change **Debugger Type** to **Managed Only**. Again on the menu bar, choose **Debug > Exceptions**, and then select **Common Language Runtime Exceptions**.

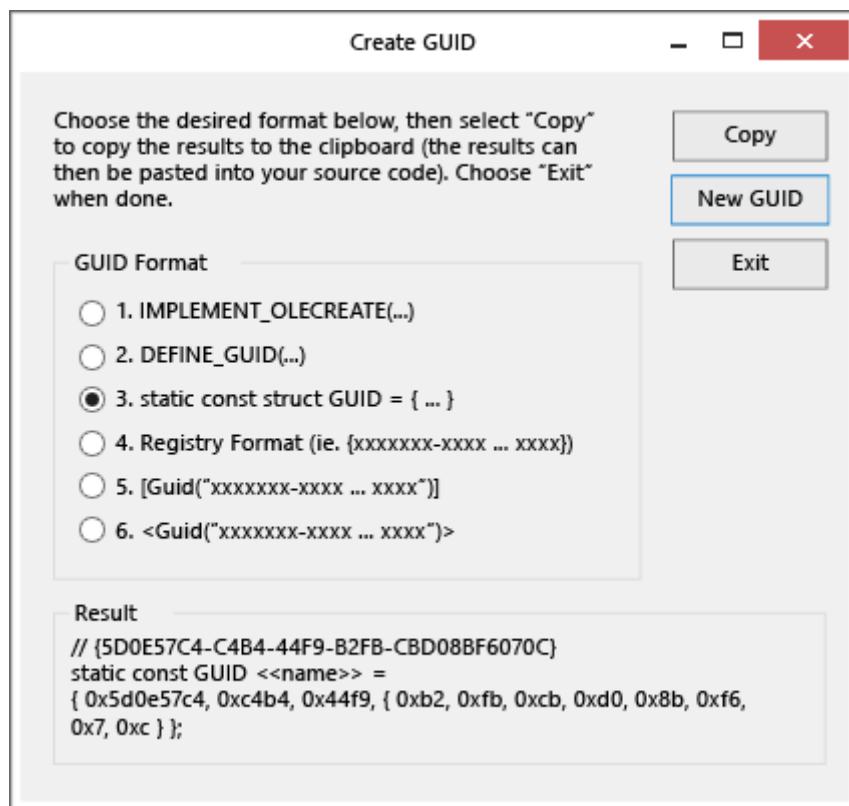
Now run the app and click the make-toast button. The debugger catches an invalid cast exception. Although it's not obvious from its message, this exception is occurring because proxies are missing for that interface.



The first step in creating a proxy and stub for a component is to add a unique ID or GUID to the interfaces. However, the GUID format to use differs depending on whether you're coding in C#, Visual Basic, or another .NET language, or in C++.

To generate GUIDs for the component's interfaces (C# and other .NET languages)

On the menu bar, choose Tools > Create GUID. In the dialog box, select 5. [Guid("xxxxxxx-xxxx...xxxx")]. Choose the New GUID button and then choose the Copy button.



Go back to the interface definition, and then paste the new GUID just before the IToaster interface, as shown in the following example. (Don't use the GUID in the example. Every unique interface should have its own GUID.)

```
C++

[Guid("FC198F74-A808-4E2A-9255-264746965B9F")]
    public interface IToaster...
```

Add a using directive for the System.Runtime.InteropServices namespace.

Repeat these steps for the IToast interface.

To generate GUIDs for the component's interfaces (C++)

On the menu bar, choose Tools > Create GUID. In the dialog box, select 3. static const struct GUID = {...}. Choose the New GUID button and then choose the Copy button.

Paste the GUID just before the IToaster interface definition. After you paste, the GUID should resemble the following example. (Don't use the GUID in the example. Every unique interface should have its own GUID.)

```
C++
```

```
// {F8D30778-9EAF-409C-BCCD-C8B24442B09B}
static const GUID <<name>> = { 0xf8d30778, 0x9eaf, 0x409c, { 0xbc, 0xcd,
0xc8, 0xb2, 0x44, 0x42, 0xb0, 0x9b } };
```

Add a using directive for Windows.Foundation.Metadata to bring GuidAttribute into scope.

Now manually convert the const GUID to a GuidAttribute so that it's formatted as shown in the following example. Notice that the curly braces are replaced with brackets and parentheses, and the trailing semicolon is removed.

C++

```
// {E976784C-AADE-4EA4-A4C0-B0C2FD1307C3}
[GuidAttribute(0xe976784c, 0xaade, 0x4ea4, 0xa4, 0xc0, 0xb0, 0xc2, 0xfd,
0x13, 0x7, 0xc3)]
public interface IToaster
{...
```

Repeat these steps for the IToast interface.

Now that the interfaces have unique IDs, we can create an IDL file by feeding the .winmd file into the winmidl command-line tool, and then generate the C source code for the proxy and stub by feeding that IDL file into the MIDL command-line tool. Visual Studio do this for us if we create post-build events as shown in the following steps.

To generate the proxy and stub source code

To add a custom post-build event, in Solution Explorer, open the shortcut menu for the ToasterComponent project and then choose Properties. In the left pane of the property pages, select Build Events, and then choose the Edit Post-build button. Add the following commands to the post-build command line. (The batch file must be called first to set the environment variables to find the winmidl tool.)

C++

```
call "$(DevEnvDir)..\vc\vcvarsall.bat" $(PlatformName)
winmidl /outdir:output "$(TargetPath)"
midl /metadata_dir "%WindowsSdkDir%References\CommonConfiguration\Neutral"
/iid "$(ProjectDir)$(TargetName)_i.c" /env win32 /h
"$(ProjectDir)$(TargetName).h" /winmd "Output\$(TargetName).winmd" /W1 /char
signed /nologo /winrt /dlldata "$(ProjectDir)dlldata.c" /proxy
"$(ProjectDir)$(TargetName)_p.c" "Output\$(TargetName).idl"
```

Important For an ARM or x64 project configuration, change the MIDL /env parameter to x64 or arm32.

To make sure the IDL file is regenerated every time the .winmd file is changed, change **Run the post-build event** to **When the build updates the project output**. The Build Events property page should resemble this:

Application Configuration: N/A Platform: N/A

Build

Build Events

Reference Paths

Signing

Code Analysis

Pre-build event command line:

Post-build event command line:

```
call "$(DevEnvDir)..\vc\vcvarsall.bat" $(PlatformName)
winmidl / outdir:output "$(TargetPath)"
midl / metadat_dir "%WindowsSdkDir%References\CommonConfig...
```

Run the post-build event:

When the build updates the project output

Rebuild the solution to generate and compile the IDL.

You can verify that MIDL correctly compiled the solution by looking for ToasterComponent.h, ToasterComponent_i.c, ToasterComponent_p.c, and dlldata.c in the ToasterComponent project directory.

To compile the proxy and stub code into a DLL

Now that you have the required files, you can compile them to produce a DLL, which is a C++ file. To make this as easy as possible, add a new project to support building the proxies. Open the shortcut menu for the ToasterApplication solution and then choose **Add > New Project**. In the left pane of the **New Project** dialog box, expand **Visual C++ > Windows > Universal Windows**, and then in the middle pane, select **DLL (UWP apps)**. (Notice that this is NOT a C++ Windows Runtime Component project.) Name the project Proxies and then choose the **OK** button. These files will be updated by the post-build events when something changes in the C# class.

By default, the Proxies project generates header .h files and C++ .cpp files. Because the DLL is built from the files produced from MIDL, the .h and .cpp files are not required. In

Solution Explorer, open the shortcut menu for them, choose **Remove**, and then confirm the deletion.

Now that the project is empty, you can add back the MIDL-generated files. Open the shortcut menu for the Proxies project, and then choose **Add > Existing Item**. In the dialog box, navigate to the ToasterComponent project directory and select these files: ToasterComponent.h, ToasterComponent_i.c, ToasterComponent_p.c, and dlldata.c files. Choose the **Add** button.

In the Proxies project, create a .def file to define the DLL exports described in dlldata.c. Open the shortcut menu for the project, and then choose **Add > New Item**. In the left pane of the dialog box, select Code and then in the middle pane, select Module-Definition File. Name the file proxies.def and then choose the **Add** button. Open this .def file and modify it to include the EXPORTS that are defined in dlldata.c:

```
C++  
  
EXPORTS  
    DllCanUnloadNow             PRIVATE  
    DllGetClassObject           PRIVATE
```

If you build the project now, it will fail. To correctly compile this project, you have to change how the project is compiled and linked. In Solution Explorer, open the shortcut menu for the Proxies project and then choose **Properties**. Change the property pages as follows.

In the left pane, select **C/C++ > Preprocessor**, and then in the right pane, select **Preprocessor Definitions**, choose the down-arrow button, and then select **Edit**. Add these definitions in the box:

```
C++  
  
WIN32;_WINDOWS
```

Under **C/C++ > Precompiled Headers**, change **Precompiled Header** to **Not Using Precompiled Headers**, and then choose the **Apply** button.

Under **Linker > General**, change **Ignore Import Library** to **Yes**, and then choose the **Apply** button.

Under **Linker > Input**, select **Additional Dependencies**, choose the down-arrow button, and then select **Edit**. Add this text in the box:

```
C++
```



```
rpcrt4.lib;runtimeobject.lib
```

Do not paste these libs directly into the list row. Use the **Edit** box to ensure that MSBuild in Visual Studio will maintain the correct additional dependencies.

When you have made these changes, choose the **OK** button in the **Property Pages** dialog box.

Next, take a dependency on the ToasterComponent project. This ensures that the Toaster will build before the proxy project builds. This is required because the Toaster project is responsible for generating the files to build the proxy.

Open the shortcut menu for the Proxies project and then choose Project Dependencies. Select the check boxes to indicate that the Proxies project depends on the ToasterComponent project, to ensure that Visual Studio builds them in the correct order.

Verify that the solution builds correctly by choosing **Build > Rebuild Solution** on the Visual Studio menu bar.

To register the proxy and stub

In the ToasterApplication project, open the shortcut menu for package.appxmanifest and then choose **Open With**. In the Open With dialog box, select **XML Text Editor** and then choose the **OK** button. We're going to paste in some XML that provides a windows.activatableClass.proxyStub extension registration and which are based on the GUIDs in the proxy. To find the GUIDs to use in the .appxmanifest file, open ToasterComponent_i.c. Find entries that resemble the ones in the following example. Also notice the definitions for IToast, IToaster, and a third interface—a typed event handler that has two parameters: a Toaster and Toast. This matches the event that's defined in the Toaster class. Notice that the GUIDs for IToast and IToaster match the GUIDs that are defined on the interfaces in the C# file. Because the typed event handler interface is autogenerated, the GUID for this interface is also autogenerated.

C++

```
MIDL_DEFINE_GUID(IID,  
IID___FITypedEventHandler_2_ToasterComponent__CToaster_ToasterComponent__CTo  
ast,0x1ecafeff,0x1ee1,0x504a,0x9a,0xf5,0xa6,0x8c,0x6f,0xb2,0xb4,0x7d);  
  
MIDL_DEFINE_GUID(IID,  
IID___x_ToasterComponent_CIToast,0xF8D30778,0x9EAF,0x409C,0xBC,0xCD,0xC8,0xB  
2,0x44,0x42,0xB0,0x9B);  
  
MIDL_DEFINE_GUID(IID,
```

```
IID__x_ToasterComponent_CIToaster,0xE976784C,0xAADE,0x4EA4,0xA4,0xC0,0xB0,0xC2,0xFD,0x13,0x07,0xC3);
```

Now we copy the GUIDs, paste them in package.appxmanifest in a node that we add and name Extensions, and then reformat them. The manifest entry resembles the following example—but again, remember to use your own GUIDs. Notice that the ClassId GUID in the XML is the same as ITypedEventHandler2. This is because that GUID is the first one that's listed in ToasterComponent_i.c. The GUIDs here are case-insensitive. Instead of manually reformatting the GUIDs for IToast and IToaster, you can go back into the interface definitions and get the GuidAttribute value, which has the correct format. In C++, there is a correctly-formatted GUID in the comment. In any case, you must manually reformat the GUID that's used for both the ClassId and the event handler.

C++

```
<Extensions> <!--Use your own GUIDs!!!-->
  <Extension Category="windows.activatableClass.proxyStub">
    <ProxyStub ClassId="1ecafeff-1ee1-504a-9af5-a68c6fb2b47d">
      <Path>Proxies.dll</Path>
      <Interface Name="IToast" InterfaceId="F8D30778-9EAF-409C-BCCD-
C8B24442B09B"/>
      <Interface Name="IToaster" InterfaceId="E976784C-AADE-4EA4-
A4C0-B0C2FD1307C3"/>
      <Interface
Name="ITypedEventHandler_2_ToasterComponent__CToaster_ToasterComponent__CToa
st" InterfaceId="1ecafeff-1ee1-504a-9af5-a68c6fb2b47d"/>
    </ProxyStub>
  </Extension>
</Extensions>
```

Paste the Extensions XML node as a direct child of the Package node, and a peer of, for example, the Resources node.

Before moving on, it's important to ensure that:

- The ProxyStub ClassId is set to the first GUID in the ToasterComponent_i.c file. Use the first GUID that's defined in this file for the classId. (This might be the same as the GUID for ITypedEventHandler2.)
- The Path is the package relative path of the proxy binary. (In this walkthrough, proxies.dll is in the same folder as ToasterApplication.winmd.)
- The GUIDs are in the correct format. (This is easy to get wrong.)
- The interface IDs in the manifest match the IIDs in ToasterComponent_i.c file.
- The interface names are unique in the manifest. Because these are not used by the system, you can choose the values. It is a good practice to choose interface names

that clearly match interfaces that you have defined. For generated interfaces, the names should be indicative of the generated interfaces. You can use the `ToasterComponent_i.c` file to help you generate interface names.

If you try to run the solution now, you will get an error that `proxies.dll` is not part of the payload. Open the shortcut menu for the **References** folder in the `ToasterApplication` project and then choose **Add Reference**. Select the check box next to the `Proxies` project. Also, make sure that the check box next to `ToasterComponent` is also selected. Choose the **OK** button.

The project should now build. Run the project and verify that you can make toast.

Related topics

- [Windows Runtime components with C++/CX](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

XAML platform

Article • 10/20/2022

This section includes topics that explain programming concepts that are generally applicable to any app that you write using C#, Visual Basic, or Visual C++ component extensions (C++/CX) and XAML for your UI definition. The programming concepts include how to use properties and events and how they apply to Universal Windows Platform (UWP) app programming. The Universal Windows Platform extends C#, Visual Basic, or C++/CX concepts of properties and their values by adding the dependency property system. Topics in this section also document the XAML language as it's used by the UWP and basic to advanced scenarios about how to use XAML to define the UI for your UWP app.

Topic	Description
XAML overview	Introduces the XAML language and concepts to the Windows Runtime app developer audience, and describes the different ways to declare objects and set attributes in XAML as it is used for creating a Windows Runtime app.
Dependency properties overview	Explains the dependency property system that is available when you write a Windows Runtime app using C++, C#, or Visual Basic along with XAML definitions for UI.
Custom dependency properties	Explains how to define and implement custom dependency properties for a Windows Runtime app using C++, C#, or Visual Basic.
Attached properties overview	Explains the concept of an attached property in XAML and provides some examples.
Custom attached properties	Explains how to implement a XAML attached property as a dependency property and how to define the accessor convention that is necessary for your attached property to be usable in XAML.
Events and routed events overview	Describes the programming concept of events in a Windows Runtime app when using C#, Visual Basic, or C++/CX as your programming language and XAML for your UI definition. You can assign handlers for events as part of the declarations for UI elements in XAML, or you can add the handlers in code. Windows Runtime supports <i>routed events</i> : certain input events and data events can be handled by objects beyond the object that fired the event. Routed events are useful when you define control templates or use pages or layout containers.

Topic	Description
UWP controls in desktop apps (XAML Islands)	Explains how to use UWP XAML controls to enhance the UI of a Windows Forms, WPF, or Win32 desktop application.

XAML overview

Article • 10/20/2022

This article introduces the XAML language and XAML concepts to the Windows Runtime app developer audience, and describes the different ways to declare objects and set attributes in XAML as it is used for creating a Windows Runtime app.

What is XAML?

Extensible Application Markup Language (XAML) is a declarative language. Specifically, XAML can initialize objects and set properties of objects using a language structure that shows hierarchical relationships between multiple objects and a backing type convention that supports extension of types. You can create visible UI elements in the declarative XAML markup. You can then associate a separate code-behind file for each XAML file that can respond to events and manipulate the objects that you originally declare in XAML.

The XAML language supports interchange of sources between different tools and roles in the development process, such as exchanging XAML sources between design tools and an interactive development environment (IDE) or between primary developers and localization developers. By using XAML as the interchange format, designer roles and developer roles can be kept separate or brought together, and designers and developers can iterate during the production of an app.

When you see them as part of your Windows Runtime app projects, XAML files are XML files with the .xaml file name extension.

Basic XAML syntax

XAML has a basic syntax that builds on XML. By definition, valid XAML must also be valid XML. But XAML also has syntax concepts that are assigned a different and more complete meaning while still being valid in XML per the XML 1.0 specification. For example, XAML supports *property element syntax*, where property values can be set within elements rather than as string values in attributes or as content. To regular XML, a XAML property element is an element with a dot in its name, so it's valid to plain XML but doesn't have the same meaning.

XAML and Visual Studio

Microsoft Visual Studio helps you to produce valid XAML syntax, both in the XAML text editor and in the more graphically oriented XAML design surface. When you write XAML for your app using Visual Studio, don't worry too much about the syntax with each keystroke. The IDE encourages valid XAML syntax by providing autocompletion hints, showing suggestions in Microsoft IntelliSense lists and dropdowns, showing UI element libraries in the **Toolbox** window, or other techniques. If this is your first experience with XAML, it might still be useful to know the syntax rules and particularly the terminology that is sometimes used to describe the restrictions or choices when describing XAML syntax in reference or other topics. The fine points of XAML syntax are covered in a separate topic, [XAML syntax guide](#).

XAML namespaces

In general programming, a namespace is an organizing concept that determines how identifiers for programming entities are interpreted. By using namespaces, a programming framework can separate user-declared identifiers from framework-declared identifiers, disambiguate identifiers through namespace qualifications, enforce rules for scoping names, and so on. XAML has its own XAML namespace concept that serves this purpose for the XAML language. Here's how XAML applies and extends the XML language namespace concepts:

- XAML uses the reserved XML attribute `xmlns` for namespace declarations. The value of the attribute is typically a Uniform Resource Identifier (URI), which is a convention inherited from XML.
- XAML uses prefixes in declarations to declare non-default namespaces, and prefix usages in elements and attributes reference that namespace.
- XAML has a concept of a default namespace, which is the namespace used when no prefix exists in a usage or declaration. The default namespace can be defined differently for each XAML programming framework.
- Namespace definitions inherit in a XAML file or construct, from parent element to child element. For example, if you define a namespace in the root element of a XAML file, all elements within that file inherit that namespace definition. If an element further into the page redefines the namespace, that element's descendants inherit the new definition.
- Attributes of an element inherit the element's namespaces. It's fairly uncommon to see prefixes on XAML attributes.

A XAML file almost always declares a default XAML namespace in its root element. The default XAML namespace defines which elements you can declare without qualifying them by a prefix. For typical Windows Runtime app projects, this default namespace contains all the built-in XAML vocabulary for the Windows Runtime that's used for UI

definitions: the default controls, text elements, XAML graphics and animations, databinding and styling support types, and so on. Most of the XAML you'll write for Windows Runtime apps will thus be able to avoid using XAML namespaces and prefixes when referring to common UI elements.

Here's a snippet that shows a template-created [Page](#) root of the initial page for an app (showing the opening tag only, and simplified). It declares the default namespace and also the x namespace (which we'll explain next).

XML

```
<Page
  x:Class="Application1.BlankPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
>
```

The XAML-language XAML namespace

One particular XAML namespace that is declared in nearly every Windows Runtime XAML file is the XAML-language namespace. This namespace includes elements and concepts that are defined by the XAML language specification. By convention, the XAML-language XAML namespace is mapped to the prefix "x". The default project and file templates for Windows Runtime app projects always define both the default XAML namespace (no prefix, just `xmlns=`) and the XAML-language XAML namespace (prefix "x") as part of the root element.

The "x" prefix/XAML-language XAML namespace contains several programming constructs that you use often in your XAML. Here are the most common ones:

Term	Description
x:Key	Sets a unique user-defined key for each resource in a XAML ResourceDictionary . The key's token string is the argument for the StaticResource markup extension, and you use this key later to retrieve the XAML resource from another XAML usage elsewhere in your app's XAML.
x:Class	Specifies the code namespace and code class name for the class that provides code-behind for a XAML page. This names the class that is created or joined by the build actions when you build your app. These build actions support the XAML markup compiler and combine your markup and code-behind when the app is compiled. You must have such a class to support code-behind for a XAML page. Window.Content in the default Windows Runtime activation model.

Term	Description
x:Name	Specifies a run-time object name for the instance that exists in run-time code after an object element defined in XAML is processed. You can think of setting x:Name in XAML as being like declaring a named variable in code. As you'll learn later, that's exactly what happens when your XAML is loaded as a component of a Windows Runtime app. Note Name is a similar property in the framework, but not all elements support it. Use x:Name for element identification whenever FrameworkElement.Name is not supported on that element type.
x:Uid	Identifies elements that should use localized resources for some of their property values. For more info on how to use x:Uid , see Quickstart: Translating UI resources .
XAML intrinsic data types	These types can specify values for simple value-types when that's required for an attribute or resource. These intrinsic types correspond to the simple value types that are typically defined as part of each programming language's intrinsic definitions. For example, you might need an object representing a true Boolean value to use in an ObjectAnimationUsingKeyFrames storyboarded visual state. For that value in XAML, you'd use the x:Boolean intrinsic type as the object element, like this: <pre><x:Boolean>True</x:Boolean></pre>

Other programming constructs in the XAML-language XAML namespace exist but are not as common.

Mapping custom types to XAML namespaces

One of the most powerful aspects of XAML as a language is that it's easy to extend the XAML vocabulary for your Windows Runtime apps. You can define your own custom types in your app's programming language and then reference your custom types in XAML markup. Support for extension through custom types is fundamentally built-in to how the XAML language works. Frameworks or app developers are responsible for creating the backing objects that XAML references. Neither frameworks nor the app developer are bound by specifications of what the objects in their vocabularies represent or do beyond the basic XAML syntax rules. (There are some expectations of what the XAML-language XAML namespace types should do, but the Windows Runtime provides all the necessary support.)

If you use XAML for types that come from libraries other than the Windows Runtime core libraries and metadata, you must declare and map a XAML namespace with a prefix. Use that prefix in element usages to reference the types that were defined in your library. You declare prefix mappings as **xmlns** attributes, typically in a root element along with the other XAML namespace definitions.

To make your own namespace definition that references custom types, you first specify the keyword **xmlns:**, then the prefix you want. The value of that attribute must contain the keyword **using:** as the first part of the value. The remainder of the value is a string token that references the specific code-backing namespace that contains your custom types, by name.

The prefix defines the markup token that is used to refer to that XAML namespace in the remainder of the markup in that XAML file. A colon character (:) goes between the prefix and the entity to be referenced within the XAML namespace.

For example, the attribute syntax to map a prefix `myTypes` to the namespace `myCompany.myTypes` is: `xmlns:myTypes="using:myCompany.myTypes"`, and a representative element usage is: `<myTypes:CustomButton/>`

For more info on mapping XAML namespaces for custom types, including special considerations for Visual C++ component extensions (C++/CX), see [XAML namespaces and namespace mapping](#).

Other XAML namespaces

You often see XAML files that define the prefixes "d" (for designer namespace) and "mc" (for markup compatibility). Generally, these are for infrastructure support or to enable scenarios in a design-time tool. For more info, see the ["Other XAML namespaces" section of the XAML namespaces topic](#).

Markup extensions

Markup extensions are a XAML language concept that is often used in the Windows Runtime XAML implementation. Markup extensions often represent some kind of "shortcut" that enables a XAML file to access a value or behavior that isn't simply declaring elements based on backing types. Some markup extensions can set properties with plain strings or with additionally nested elements, with the goal of streamlining the syntax or the factoring between different XAML files.

In XAML attribute syntax, curly braces "{" and "}" indicate a XAML markup extension usage. This usage directs the XAML processing to escape from the general treatment of treating attribute values as either a literal string or a directly string-convertible value. Instead, a XAML parser calls code that provides behavior for that particular markup extension, and that code provides an alternate object or behavior result that the XAML parser needs. Markup extensions can have arguments, which follow the markup extension name and are also contained within the curly braces. Typically, an evaluated

markup extension provides an object return value. During parsing, that return value is inserted into the position in the object tree where the markup extension usage was in the source XAML.

Windows Runtime XAML supports these markup extensions that are defined under the default XAML namespace and are understood by the Windows Runtime XAML parser:

- [{x:Bind}](#): supports data binding, which defers property evaluation until run-time by executing special-purpose code, which it generates at compile-time. This markup extension supports a wide range of arguments.
- [{Binding}](#): supports data binding, which defers property evaluation until run-time by executing general-purpose runtime object inspection. This markup extension supports a wide range of arguments.
- [{StaticResource}](#): supports referencing resource values that are defined in a [ResourceDictionary](#). These resources can be in a different XAML file but must ultimately be findable by the XAML parser at load time. The argument of a `{StaticResource}` usage identifies the key (the name) for a keyed resource in a [ResourceDictionary](#).
- [{ThemeResource}](#): similar to [{StaticResource}](#) but can respond to run-time theme changes. `{ThemeResource}` appears quite often in the Windows Runtime default XAML templates, because most of these templates are designed for compatibility with the user switching the theme while the app is running.
- [{TemplateBinding}](#): a special case of [{Binding}](#) that supports control templates in XAML and their eventual usage at run time.
- [{RelativeSource}](#): enables a particular form of template binding where values come from the templated parent.
- [{CustomResource}](#): for advanced resource lookup scenarios.

Windows Runtime also supports the [{x:Null}](#) markup extension. You use this to set [Nullable](#) values to `null` in XAML. For example you might use this in a control template for a [CheckBox](#), which interprets `null` as an indeterminate check state (triggering the "Indeterminate" visual state).

A markup extension generally returns an existing instance from some other part of the object graph for the app or defers a value to run time. Because you can use a markup extension as an attribute value, and that's the typical usage, you often see markup extensions providing values for reference-type properties that might have otherwise required a property element syntax.

For example, here's the syntax for referencing a reusable [Style](#) from a [ResourceDictionary](#): `<Button Style="{StaticResource SearchButtonStyle}"/>`. A [Style](#) is a reference type, not a simple value, so without the `{StaticResource}` usage, you would've

needed a `<Button.Style>` property element and a `<Style>` definition within it to set the [FrameworkElement.Style](#) property.

By using markup extensions, every property that is settable in XAML is potentially settable in attribute syntax. You can use attribute syntax to provide reference values for a property even if it doesn't otherwise support an attribute syntax for direct object instantiation. Or you can enable specific behavior that defers the general requirement that XAML properties be filled by value types or by newly created reference types.

To illustrate, the next XAML example sets the value of the [FrameworkElement.Style](#) property of a [Border](#) by using attribute syntax. The [FrameworkElement.Style](#) property takes an instance of the [Windows.UI.Xaml.Style](#) class, a reference type that by default could not be created using an attribute syntax string. But in this case, the attribute references a particular markup extension, [StaticResource](#). When that markup extension is processed, it returns a reference to a **Style** element that was defined earlier as a keyed resource in a resource dictionary.

XML

```
<Canvas.Resources>
  <Style TargetType="Border" x:Key="PageBackground">
    <Setter Property="BorderBrush" Value="Blue"/>
    <Setter Property="BorderThickness" Value="5"/>
  </Style>
</Canvas.Resources>
...
<Border Style="{StaticResource PageBackground}">
  ...
</Border>
```

You can nest markup extensions. The innermost markup extension is evaluated first.

Because of markup extensions, you need special syntax for a literal "{" value in an attribute. For more info see [XAML syntax guide](#).

Events

XAML is a declarative language for objects and their properties, but it also includes a syntax for attaching event handlers to objects in the markup. The XAML event syntax can then integrate the XAML-declared events through the Windows Runtime programming model. You specify the name of the event as an attribute name on the object where the event is handled. For the attribute value, you specify the name of an event-handler function that you define in code. The XAML processor uses this name to create a delegate representation in the loaded object tree, and adds the specified

handler to an internal handler list. Nearly all Windows Runtime apps are defined by both markup and code-behind sources.

Here's a simple example. The [Button](#) class supports an event named [Click](#). You can write a handler for **Click** that runs code that should be invoked after the user clicks the **Button**. In XAML, you specify **Click** as an attribute on the **Button**. For the attribute value, provide a string that is the method name of your handler.

XML

```
<Button Click="showUpdatesButton_Click">Show updates</Button>
```

When you compile, the compiler now expects that there will be a method named `showUpdatesButton_Click` defined in the code-behind file, in the namespace declared in the XAML page's `x:Class` value. Also, that method must satisfy the delegate contract for the [Click](#) event. For example:

C#

```
namespace App1
{
    public sealed partial class MainPage: Page {
        ...
        private void showUpdatesButton_Click (object sender, RoutedEventArgs
e) {
            //your code
        }
    }
}
```

Within a project, the XAML is written as a .xaml file, and you use the language you prefer (C#, Visual Basic, C++/CX) to write a code-behind file. When a XAML file is markup-compiled as part of a build action for the project, the location of the XAML code-behind file for each XAML page is identified by specifying a namespace and class as the `x:Class` attribute of the root element of the XAML page. For more info on how these mechanisms work in XAML and how they relate to the programming and application models, see [Events and routed events overview](#).

ⓘ Note

For C++/CX there are two code-behind files: one is a header (.xaml.h) and the other is implementation (.xaml.cpp). The implementation references the header, and it's technically the header that represents the entry point for the code-behind connection.

Resource dictionaries

Creating a [ResourceDictionary](#) is a common task that is usually accomplished by authoring a resource dictionary as an area of a XAML page or a separate XAML file. Resource dictionaries and how to use them is a larger conceptual area that is outside the scope of this topic. For more info see [ResourceDictionary and XAML resource references](#).

XAML and XML

The XAML language is fundamentally based on the XML language. But XAML extends XML significantly. In particular it treats the concept of schema quite differently because of its relationship to the backing type concept, and adds language elements such as attached members and markup extensions. **xml:lang** is valid in XAML, but influences runtime rather than parse behavior, and is typically aliased to a framework-level property. For more info, see [FrameworkElement.Language](#). **xml:base** is valid in markup but parsers ignore it. **xml:space** is valid, but is only relevant for scenarios described in the [XAML and whitespace](#) topic. The **encoding** attribute is valid in XAML. Only UTF-8 and UTF-16 encodings are supported. UTF-32 encoding is not supported.

Case sensitivity in XAML

XAML is case-sensitive. This is another consequence of XAML being based on XML, which is case-sensitive. The names of XAML elements and attributes are case-sensitive. The value of an attribute is potentially case-sensitive; this depends on how the attribute value is handled for particular properties. For example, if the attribute value declares a member name of an enumeration, the built-in behavior that type-converts a member name string to return the enumeration member value is not case-sensitive. In contrast, the value of the **Name** property, and utility methods for working with objects based on the name that the **Name** property declares, treat the name string as case-sensitive.

XAML namespaces

The XAML language defines a concept of a XAML namespace. The XAML namespace concept influences how XAML processors should treat the value of **x>Name** or **Name** applied to XAML elements, particularly the scopes in which names should be relied upon to be unique identifiers. XAML namespaces are covered in more detail in a separate topic; see [XAML namespaces](#).

The role of XAML in the development process

XAML plays several important roles in the app development process.

- XAML is the primary format for declaring an app's UI and elements in that UI, if you are programming using C#, Visual Basic or C++/CX. Typically at least one XAML file in your project represents a page metaphor in your app for the initially displayed UI. Additional XAML files might declare additional pages for navigation UI. Other XAML files can declare resources, such as templates or styles.
- You use the XAML format for declaring styles and templates applied to controls and UI for an app.
- You might use styles and templates either for templating existing controls, or if you define a control that supplies a default template as part of a control package. When you use it to define styles and templates, the relevant XAML is often declared as a discrete XAML file with a [ResourceDictionary](#) root.
- XAML is the common format for designer support of creating app UI and exchanging the UI design between different designer apps. Most notably, XAML for the app can be interchanged between different XAML design tools (or design windows within tools).
- Several other technologies also define the basic UI in XAML. In relationship to Windows Presentation Foundation (WPF) XAML and Microsoft Silverlight XAML, the XAML for Windows Runtime uses the same URI for its shared default XAML namespace. The XAML vocabulary for Windows Runtime overlaps significantly with the XAML-for-UI vocabulary also used by Silverlight and to a slightly lesser extent by WPF. Thus, XAML promotes an efficient migration pathway for UI originally defined for precursor technologies that also used XAML.
- XAML defines the visual appearance of a UI, and an associated code-behind file defines the logic. You can adjust the UI design without making changes to the logic in code-behind. XAML simplifies the workflow between designers and developers.
- Because of the richness of the visual designer and design surface support for the XAML language, XAML supports rapid UI prototyping in the early development phases.

Depending on your own role in the development process, you might not interact with XAML much. The degree to which you do interact with XAML files also depends on which development environment you are using, whether you use interactive design environment features such as toolboxes and property editors, and the scope and purpose of your Windows Runtime app. Nevertheless, it is likely that during development of the app, you will be editing a XAML file at the element level using a text or XML editor. Using this info, you can confidently edit XAML in a text or XML

representation and maintain the validity of that XAML file's declarations and purpose when it is consumed by tools, markup compile operations, or the run-time phase of your Windows Runtime app.

Optimize your XAML for load performance

Here are some tips for defining UI elements in XAML using best practices for performance. Many of these tips relate to using XAML resources, but are listed here in the general XAML overview for convenience. For more info about XAML resources see [ResourceDictionary and XAML resource references](#). For some more tips on performance, including XAML that purposely demonstrates some of the poor performance practices that you should avoid in your XAML, see [Optimize your XAML markup](#).

- If you use the same color brush often in your XAML, define a [SolidColorBrush](#) as a resource rather than using a named color as an attribute value each time.
- If you use the same resource on more than one UI page, consider defining it in [Resources](#) rather than on each page. Conversely, if only one page uses a resource, don't define it in **Application.Resources** and instead define it only for the page that needs it. This is good both for XAML factoring while designing your app and for performance during XAML parsing.
- For resources that your app packages, check for unused resources (a resource that has a key, but there's no [StaticResource](#) reference in your app that uses it). Remove these from your XAML before you release your app.
- If you're using separate XAML files that provides design resources ([MergedDictionaries](#)), consider commenting or removing unused resources from these files. Even if you have a shared XAML starting point that you're using in more than one app or that provides common resources for all your app, it's still your app that packages the XAML resources each time, and potentially has to load them.
- Don't define UI elements you don't need for composition, and use the default control templates whenever possible (these templates have already been tested and verified for load performance).
- Use containers such as [Border](#) rather than deliberate overdraws of UI elements. Basically, don't draw the same pixel multiple times. For more info on overdraw and how to test for it, see [DebugSettings.IsOverdrawHeatMapEnabled](#).
- Use the default items templates for [ListView](#) or [GridView](#); these have special **Presenter** logic that solves performance issues when building the visual tree for large numbers of list items.

Debug XAML

Because XAML is a markup language, some of the typical strategies for debugging within Microsoft Visual Studio are not available. For example, there is no way to set a breakpoint within a XAML file. However, there are other techniques that can help you debug issues with UI definitions or other XAML markup while you're still developing your app.

When there are problems with a XAML file, the most typical result is that some system or your app will throw a XAML parse exception. Whenever there is a XAML parse exception, the XAML loaded by the XAML parser failed to create a valid object tree. In some cases, such as when the XAML represents the first "page" of your application that is loaded as the root visual, the XAML parse exception is not recoverable.

XAML is often edited within an IDE such as Visual Studio and one of its XAML design surfaces. Visual Studio can often provide design-time validation and error checking of a XAML source as you edit it. For example it might display "squiggles" in the XAML text editor as soon as you type a bad attribute value, and you won't even have to wait for a XAML compile pass to see that something's wrong with your UI definition.

Once the app actually runs, if any XAML parse errors have gone undetected at design time, these are reported by the common language runtime (CLR) as a [XamlParseException](#). For more info on what you might be able to do for a run-time [XamlParseException](#), see [Exception handling for Windows Runtime apps in C# or Visual Basic](#).

ⓘ Note

Apps that use C++/CX for code don't get the specific [XamlParseException](#). But the message in the exception clarifies that the source of the error is XAML-related, and includes context info such as line numbers in a XAML file, just like [XamlParseException](#) does.

For more info on debugging a Windows Runtime app, see [Start a debug session](#).

XAML syntax guide

Article • 10/20/2022

We explain XAML syntax rules and the terminology that describes the restrictions or choices available for XAML syntax. You'll find this topic useful if you are new to using the XAML language, you want a refresher on the terminology or parts of syntax, or you are curious about how the XAML language works and want more background and context.

XAML is XML

Extensible Application Markup Language (XAML) has a basic syntax that builds on XML, and by definition valid XAML must be valid XML. But XAML also has its own syntax concepts that extend XML. A given XML entity might be valid in plain XML, but that syntax might have a different and more complete meaning as XAML. This topic explains these XAML syntax concepts.

XAML vocabularies

One area where XAML differs from most XML usages is that XAML is not typically enforced with a schema, such as an XSD file. That's because XAML is intended to be extensible, that's what the "X" in the acronym XAML means. Once XAML is parsed, the elements and attributes you reference in XAML are expected to exist in some backing code representation, either in the core types defined by the Windows Runtime, or in types that extend or are based off the Windows Runtime. The SDK documentation sometimes refers to the types that are already built-in to the Windows Runtime and can be used in XAML as being the *XAML vocabulary* for the Windows Runtime. Microsoft Visual Studio helps you to produce markup that's valid within this XAML vocabulary. Visual Studio can also include your custom types for XAML usage so long as the source of those types is referenced correctly in the project. For more info about XAML and custom types, see [XAML namespaces and namespace mapping](#).

Declaring objects

Programmers often think in terms of objects and members, whereas a markup language is conceptualized as elements and attributes. In the most basic sense, an element that you declare in XAML markup becomes an object in a backing runtime object representation. To create a run-time object for your app, you declare a XAML element in the XAML markup. The object is created when the Windows Runtime loads your XAML.

A XAML file always has exactly one element serving as its root, which declares an object that will be the conceptual root of some programming structure such as a page, or the object graph of the entire run-time definition of an application.

In terms of XAML syntax, there are three ways to declare objects in XAML:

- **Directly, using object element syntax:** This uses opening and closing tags to instantiate an object as an XML-form element. You can use this syntax to declare root objects or to create nested objects that set property values.
- **Indirectly, using attribute syntax:** This uses an inline string value that has instructions for how to create an object. The XAML parser uses this string to set the value of a property to a newly created reference value. Support for it is limited to certain common objects and properties.
- Using a markup extension.

This does not mean that you always have the choice of any syntax for object creation in a XAML vocabulary. Some objects can be created only by using object element syntax. Some objects can be created only by being initially set in an attribute. In fact, objects that can be created with either object element or attribute syntax are comparatively rare in XAML vocabularies. Even if both syntax forms are possible, one of the syntaxes will be more common as a matter of style. There are also techniques you can use in XAML to reference existing objects rather than creating new values. The existing objects might be defined either in other areas of XAML, or might exist implicitly through some behavior of the platform and its application or programming models.

Declaring an object by using object element syntax

To declare an object with object element syntax, you write tags like this: `<objectName>`
`</objectName>`, where *objectName* is the type name for the object you want to instantiate. Here's object element usage to declare a [Canvas](#) object:

XML

```
<Canvas>  
</Canvas>
```

If the object does not contain other objects, you can declare the object element by using one self-closing tag instead of an opening/closing pair: `<Canvas />`

Containers

Many objects used as UI elements, such as [Canvas](#), can contain other objects. These are sometimes referred to as containers. The following example shows a **Canvas** container that contains one element, a [Rectangle](#).

XML

```
<Canvas>
  <Rectangle />
</Canvas>
```

Declaring an object by using attribute syntax

Because this behavior is tied to property setting, we'll talk about this more in upcoming sections.

Initialization text

For some objects you can declare new values using inner text that's used as initialization values for construction. In XAML, this technique and syntax is called *initialization text*. Conceptually, initialization text is similar to calling a constructor that has parameters. Initialization text is useful for setting initial values of certain structures.

You often use an object element syntax with initialization text if you want a structure value with an **x:Key**, so it can exist in a [ResourceDictionary](#). You might do this if you share that structure value among multiple target properties. For some structures, you can't use attribute syntax to set the structure's values: initialization text is the only way to produce a useful and shareable [CornerRadius](#), [Thickness](#), [GridLength](#) or [Color](#) resource.

This abbreviated example uses initialization text to specify values for a [Thickness](#), in this case specifying values that set both **Left** and **Right** to 20, and both **Top** and **Bottom** to 10. This example shows the **Thickness** created as a keyed resource, and then the reference to that resource. For more info on [Thickness](#) initialization text, see [Thickness](#).

XML

```
<UserControl ...>
  <UserControl.Resources>
    <Thickness x:Key="TwentyTenThickness">20,10</Thickness>
    ...
  </UserControl.Resources>
  ...
  <Grid Margin="{StaticResource TwentyTenThickness}">
    ...
```

```
</Grid>  
</UserControl ...>
```

Note Some structures can't be declared as object elements. Initialization text isn't supported and they can't be used as resources. You must use an attribute syntax in order to set properties to these values in XAML. These types are: [Duration](#), [RepeatBehavior](#), [Point](#), [Rect](#) and [Size](#).

Setting properties

You can set properties on objects that you declared by using object element syntax. There are multiple ways to set properties in XAML:

- By using attribute syntax.
- By using property element syntax.
- By using element syntax where the content (inner text or child elements) is setting the XAML content property of an object.
- By using a collection syntax (which is usually the implicit collection syntax).

As with object declaration, this list doesn't imply that any property could be set with each of the techniques. Some properties support only one of the techniques. Some properties support more than one form; for example, there are properties that can use property element syntax, or attribute syntax. What's possible depends both on the property and on the object type that the property uses. In the Windows Runtime API reference, you'll see the XAML usages you can use in the **Syntax** section. Sometimes there is an alternative usage that would work but would be more verbose. Those verbose usages aren't always shown because we are trying to show you the best practices or the real world scenarios for using that property in XAML. Guidance for XAML syntax is provided in the **XAML Usage** sections of reference pages for properties that can be set in XAML.

Some properties on objects cannot be set in XAML by any means, and can only be set using code. Usually these are properties that are more appropriate to work with in the code-behind, not in XAML.

A read-only property cannot be set in XAML. Even in code, the owning type would have to support some other way to set it, like a constructor overload, helper method, or calculated property support. A calculated property relies on the values of other settable properties plus sometimes an event with built-in handling; these features are available in the dependency property system. For more info on how dependency properties are useful for calculated property support, see [Dependency properties overview](#).

Collection syntax in XAML gives an appearance that you are setting a read-only property, but in fact you are not. See "[Collection Syntax](#)" later in this topic.

Setting a property by using attribute syntax

Setting an attribute value is the typical means by which you set a property value in a markup language, for example in XML or HTML. Setting XAML attributes is similar to how you set attribute values in XML. The attribute name is specified at any point within the tags following the element name, separated from element name by at least one whitespace. The attribute name is followed by an equals sign. The attribute value is contained within a pair of quotes. The quotes can be either double quotes or single quotes so long as they match and enclose the value. The attribute value itself must be expressible as a string. The string often contains numerals, but to XAML, all attribute values are string values until the XAML parser gets involved and does some basic value conversion.

This example uses attribute syntax for four attributes to set the [Name](#), [Width](#), [Height](#), and [Fill](#) properties of a [Rectangle](#) object.

XML

```
<Rectangle Name="rectangle1" Width="100" Height="100" Fill="Blue" />
```

Setting a property by using property element syntax

Many properties of an object can be set by using property element syntax. A property element looks like this: `<object.property>`.

To use property element syntax, you create XAML property elements for the property that you want to set. In standard XML, this element would just be considered an element that has a dot in its name. However, in XAML, the dot in the element name identifies the element as a property element, with *property* expected to be a member of *object* in a backing object model implementation. To use property element syntax, it must be possible to specify an object element in order to "fill" the property element tags. A property element will always have some content (single element, multiple elements, or inner text); there's no point in having a self-closing property element.

In the following grammar, *property* is the name of the property that you want to set and *propertyValueAsObjectElement* is a single object element, that's expected to satisfy the value type requirements of the property.

`<object>`

`<object .property>`

propertyValueAsObjectElement

`</object .property>`

`</object>`

The following example uses property element syntax to set the **Fill** of a **Rectangle** with a **SolidColorBrush** object element. (Within the **SolidColorBrush**, **Color** is set as an attribute.) The parsed result of this XAML is identical to the previous XAML example that set **Fill** using attribute syntax.

XML

```
<Rectangle
  Name="rectangle1"
  Width="100"
  Height="100"
>
  <Rectangle.Fill>
    <SolidColorBrush Color="Blue"/>
  </Rectangle.Fill>
</Rectangle>
```

XAML vocabularies and object-oriented programming

Properties and events as they appear as XAML members of a Windows Runtime XAML type are often inherited from base types. Consider this example: `<Button Background="Blue" .../>`. The **Background** property is not an immediately declared property on the **Button** class. Instead, **Background** is inherited from the base **Control** class. In fact, if you look at the reference topic for **Button** you'll see that the members lists contain at least one inherited member from each of a chain of successive base classes: **ButtonBase**, **Control**, **FrameworkElement**, **UIElement**, **DependencyObject**. In the **Properties** list, all the read-write properties and collection properties are inherited in a XAML vocabulary sense. Events (like the various **UIElement** events) are inherited too.

If you use the Windows Runtime reference for XAML guidance, the element name that's shown in a syntax or even in example code is sometimes for the type that originally defines the property, because that reference topic is shared by all the possible types that inherit it from a base class. If you use Visual Studio's IntelliSense for XAML in the XML editor, the IntelliSense and its drop-downs do a great job of coalescing the inheritance and providing an accurate list of attributes that are available for setting once you've started with an object element for a class instance.

XAML content properties

Some types define one of their properties such that the property enables a XAML content syntax. For the XAML content property of a type, you can omit the property element for that property when specifying it in XAML. Or, you can set the property to an inner text value by providing that inner text directly within the owning type's object element tags. XAML content properties support straightforward markup syntax for that property and makes the XAML more human-readable by reducing the nesting.

If a XAML content syntax is available, that syntax will be shown in the "XAML" sections of **Syntax** for that property in the Windows Runtime reference documentation. For example, the [Child](#) property page for [Border](#) shows XAML content syntax instead of property element syntax to set the single-object **Border.Child** value of a **Border**, like this:

XML

```
<Border>
  <Button .../>
</Border>
```

If the property that is declared as the XAML content property is the **Object** type, or is type **String**, then the XAML content syntax supports what's basically inner text in the XML document model: a string between the opening and closing object tags. For example, the [Text](#) property page for [TextBlock](#) shows XAML content syntax that has an inner text value to set **Text**, but the string "Text" never appears in the markup. Here's an example usage:

XML

```
<TextBlock>Hello!</TextBlock>
```

If a XAML content property exists for a class, that's indicated in the reference topic for the class, in the "Attributes" section. Look for the value of the [ContentPropertyAttribute](#). This attribute uses a named field "Name". The value of "Name" is the name of the property of that class that is the XAML content property. For example, on the [Border](#) reference page, you'll see this: `ContentProperty("Name=Child")`.

One important XAML syntax rule we should mention is that you can't intermix the XAML content property and other property elements you set on the element. The XAML content property must be set entirely before any property elements, or entirely after. For example this is invalid XAML:

syntax


```
<StackPanel>
  <Button>This example</Button>
  <StackPanel.Resources>
    <SolidColorBrush x:Key="BlueBrush" Color="Blue"/>
  </StackPanel.Resources>
  <Button>... is illegal XAML</Button>
</StackPanel>
```

Collection syntax

All of the syntaxes shown thus far are setting properties to single objects. But many UI scenarios require that a given parent element can have multiple child elements. For example, a UI for an input form needs several text box elements, some labels, and perhaps a "Submit" button. Still, if you were to use a programming object model to access these multiple elements, they would typically be items in a single collection property, rather than each item being the value of different properties. XAML supports multiple child elements as well as supporting a typical backing collection model by treating properties that use a collection type as implicit, and performing special handling for any child elements of a collection type.

Many collection properties are also identified as the XAML content property for the class. The combination of implicit collection processing and XAML content syntax is frequently seen in types used for control compositing, such as panels, views, or items controls. For example, the following example shows the simplest possible XAML for compositing two peer UI elements within a [StackPanel](#).

XML

```
<StackPanel>
  <TextBlock>Hello</TextBlock>
  <TextBlock>World</TextBlock>
</StackPanel>
```

The mechanism of XAML collection syntax

It might at first appear that XAML is enabling a "set" of the read-only collection property. In reality, what XAML enables here is adding items to an existing collection. The XAML language and XAML processors implementing XAML support rely on a convention in backing collection types to enable this syntax. Typically there is a backing property such as an indexer or **Items** property that refers to specific items of the collection. Generally, that property is not explicit in the XAML syntax. For collections, the underlying mechanism for XAML parsing is not a property, but a method: specifically,

the **Add** method in most cases. When the XAML processor encounters one or more object elements within a XAML collection syntax, each such object is first created from an element, then each new object is added in order to the containing collection by calling the collection's **Add** method.

When a XAML parser adds items to a collection, it is the logic of the **Add** method that determines whether a given XAML element is a permissible item child of the collection object. Many collection types are strongly typed by the backing implementation, meaning that the input parameter of **Add** expects that whatever is passed must be a type match with the **Add** parameter type.

For collection properties, be careful about when you try to specify the collection explicitly as an object element. A XAML parser will create a new object whenever it encounters an object element. If the collection property you're trying to use is read-only, this can throw a XAML parse exception. Just use the implicit collection syntax, and you won't see that exception.

When to use attribute or property element syntax

All properties that support being set in XAML will support attribute or property element syntax for direct value setting, but potentially will not support either syntax interchangeably. Some properties do support either syntax, and some properties support additional syntax options like a XAML content property. The type of XAML syntax supported by a property depends on the type of object that the property uses as its property type. If the property type is a primitive type, such as a double (float or decimal), integer, Boolean, or string, the property always supports attribute syntax.

You can also use attribute syntax to set a property if the object type you use to set that property can be created by processing a string. For primitives, this is always the case, the type conversion is built in to the parser. However, certain other object types can also be created by using a string specified as an attribute value, rather than an object element within a property element. For this to work, there has to be an underlying type conversion, supported either by that particular property or supported generally for all values that use that property type. The string value of the attribute is used to set properties that are important for the initialization of the new object value. Potentially, a specific type converter can also create different subclasses of a common property type, depending on how it uniquely processes information in the string. Object types that support this behavior will have a special grammar listed in the syntax section of the reference documentation. As an example, the XAML syntax for [Brush](#) shows how an

attribute syntax can be used to create a new [SolidColorBrush](#) value for any property of type **Brush** (and there are many **Brush** properties in Windows Runtime XAML).

XAML parsing logic and rules

Sometime's it's informative to read the XAML in a similar way to how a XAML parser must read it: as a set of string tokens encountered in a linear order. A XAML parser must interpret these tokens under a set of rules that are part of the definition of how XAML works.

Setting an attribute value is the typical means by which you set a property value in a markup language, for example in XML or HTML. In the following syntax, *objectName* is the object you want to instantiate, *propertyName* is the name of the property that you want to set on that object, and *propertyValue* is the value to set.

XML

```
<objectName propertyName="propertyValue" .../>
```

-or-

```
<objectName propertyName="propertyValue">
```

```
...<!--element children -->
```

```
</objectName>
```

Either syntax enables you to declare an object and set a property on that object. Although the first example is a single element in markup, there are actually discrete steps here with regard to how a XAML processor parses this markup.

First, the presence of the object element indicates that a new *objectName* object must be instantiated. Only after such an instance exists can the instance property *propertyName* be set on it.

Another rule of XAML is that attributes of an element must be able to be set in any order. For example, there's no difference between `<Rectangle Height="50" Width="100" />` and `<Rectangle Width="100" Height="50" />`. Which order you use is a matter of style.

Note XAML designers often promote ordering conventions if you use design surfaces other than the XML editor, but you can freely edit that XAML later, to reorder the attributes or introduce new ones.

Attached properties

XAML extends XML by adding a syntax element known as an *attached property*. Similar to the property element syntax, the attached property syntax contains a dot, and the dot holds special meaning to XAML parsing. Specifically, the dot separates the provider of the attached property, and the property name.

In XAML, you set attached properties by using the syntax *AttachedPropertyProvider.PropertyName*. Here is an example of how you can set the attached property [Canvas.Left](#) in XAML:

XML

```
<Canvas>
  <Button Canvas.Left="50">Hello</Button>
</Canvas>
```

You can set the attached property on elements that don't have a property of that name in the backing type, and in that way they function somewhat like a global property, or an attribute defined by a different XML namespace like the `xml:space` attribute.

In Windows Runtime XAML you'll see attached properties that support these scenarios:

- Child elements can inform parent container panels how they should behave in layout: [Canvas](#), [Grid](#), [VariableSizedWrapGrid](#).
- Control usages can influence behavior of an important control part that comes from the control template: [ScrollViewer](#), [VirtualizingStackPanel](#).
- Using a service that's available in a related class, where the service and the class that uses it don't share inheritance: [Typography](#), [VisualStateManager](#), [AutomationProperties](#), [ToolTipService](#).
- Animation targeting: [Storyboard](#).

For more info, see [Attached properties overview](#).

Literal "{" values

Because the opening brace symbol { is the opening of the markup extension sequence, you use an escape sequence to specify a literal string value that starts with "{". The escape sequence is "{}". For example, to specify a string value that is a single opening brace, specify the attribute value as "{}{". You can also use the alternative quotation marks (for example, a ' within an attribute value delimited by "") to provide a "{" value as a string.

Note "\" also works if it's inside a quoted attribute.

Enumeration values

Many properties in the Windows Runtime API use enumerations as values. If the member is a read-write property you can set such a property by providing an attribute value. You identify which enumeration value to use as the value of the property by using the unqualified name of the constant name. For example here's how to set [UIElement.Visibility](#) in XAML: `<Button Visibility="Visible"/>`. Here the "Visible" as a string is directly mapped to a named constant of the [Visibility](#) enumeration, **Visible**.

- Don't use a qualified form, it won't work. For example, this is invalid XAML: `<Button Visibility="Visibility.Visible"/>`.
- Don't use the value of the constant. In other words, don't rely on the integer value of the enumeration that's there explicitly or implicitly depending on how the enumeration was defined. Although it might appear to work, it's a bad practice either in XAML or in code because you're relying on what could be a transient implementation detail. For example, don't do this: `<Button Visibility="1"/>`.

Note In reference topics for APIs that use XAML and use enumerations, click the link to the enumeration type in the **Property value** section of **Syntax**. This links to the enumeration page where you can discover the named constants for that enumeration.

Enumerations can be flagwise, meaning that they are attributed with **FlagsAttribute**. If you need to specify a combination of values for a flagwise enumeration as a XAML attribute value, use the name of each enumeration constant, with a comma (,) between each name, and no intervening space characters. Flagwise attributes aren't common in the Windows Runtime XAML vocabulary, but [ManipulationModes](#) is an example where setting a flagwise enumeration value in XAML is supported.

Interfaces in XAML

In rare cases you'll see a XAML syntax where the type of a property is an interface. In the XAML type system, a type that implements that interface is acceptable as a value when parsed. There must be a created instance of such a type available to serve as the value. You'll see an interface used as a type in the XAML syntax for [Command](#) and [CommandParameter](#) properties of [ButtonBase](#). These properties support Model-View-ViewModel (MVVM) design patterns where the **ICommand** interface is the contract for how the views and models interact.